



Laboratório 6: Pré-processamento de dados de imagem

1 Introdução

1.1 Sobre Este Laboratório

O pré-processamento de imagens tem como objetivo remover informações irrelevantes das imagens, restaurar informações úteis reais, aumentar a detectabilidade de informações relevantes e simplificar os dados o máximo possível. Esse processo melhora a confiabilidade da extração de características, segmentação, correspondência e reconhecimento de imagens.

Este experimento utiliza a biblioteca de processamento de imagens OpenCV para implementar operações básicas de pré-processamento de imagens, incluindo:

- Operações básicas com imagens;
- Conversão de espaço de cores;
- Transformações geométricas;
- Transformações em escala de cinza;
- Processamento morfológico;
- Filtragem de imagens.

1.2 Objetivos

Este experimento implementa técnicas de pré-processamento de imagens apresentadas na teoria utilizando a biblioteca OpenCV da linguagem Python. Por meio deste experimento, você compreenderá como utilizar o OpenCV para pré-processamento de imagens. Ao observar alterações em dados reais de imagens, será possível aprofundar a compreensão das técnicas de pré-processamento de imagens. Este experimento auxiliará no entendimento e domínio dos métodos e técnicas de desenvolvimento de pré-processamento de imagens utilizando Python.

1.3 Configuração do Ambiente

Recomenda-se instalar: Python 3.6 ou superior, OpenCV, NumPy, Matplotlib.

1.4 Procedimento

1.4.1 Operações Básicas com Imagens

Observação: Você pode carregar imagens do computador local para o Notebook para utilização nos códigos desta seção.

Dados necessários para o experimento: <https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIPI-AI%20EI%20Developer/V2.5/chapter2/cv.zip>

Etapa 1 – Ler e exibir imagens.

```
import cv2
import matplotlib.pyplot as plt

%matplotlib inline

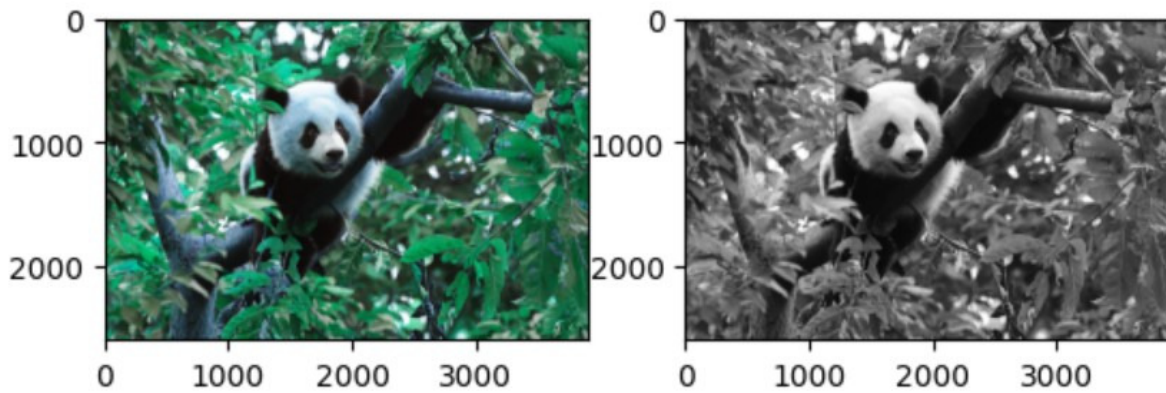
# Leitura da imagem. Se o segundo parâmetro for 1, a imagem colorida é
# carregada.
# Se for 0, a imagem em escala de cinza é carregada.
im = cv2.imread(r"panda.jpg",1)
im_gray = cv2.imread(r"panda.jpg",0)

# Cria uma figura.
plt.figure()

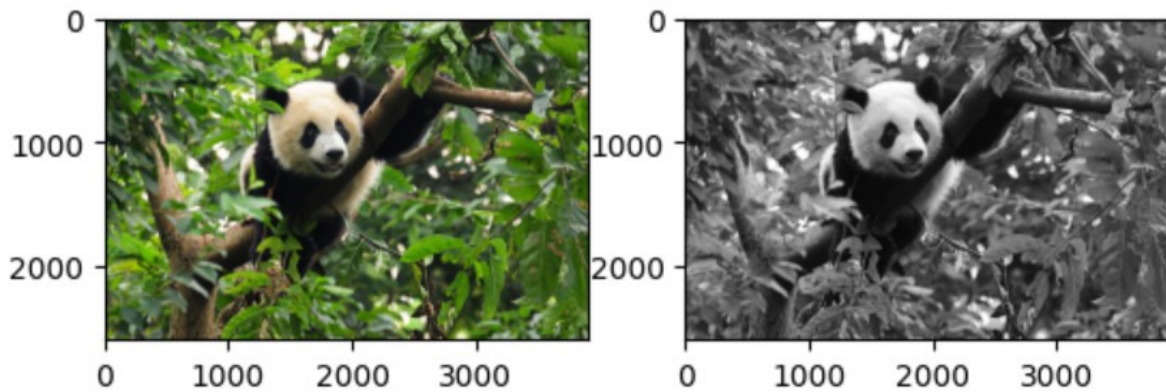
# Quando imagens são exibidas usando
# Matplotlib, as cores podem ficar incorretas.
plt.subplot(1,2,1),plt.imshow(im)
plt.subplot(1,2,2),plt.imshow(im_gray,'gray')
plt.show()

# Isso ocorre porque o OpenCV utiliza o padrão BGR, enquanto o
# Matplotlib utiliza RGB.
im_RGB = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2RGB
)
plt.subplot(1,2,1),plt.imshow(im_RGB)
plt.subplot(1,2,2),plt.imshow(im_gray,'gray')
```

Resultado:



(<AxesSubplot:>, <matplotlib.image.AxesImage at 0x7fe831619d90>)



Etapa 2 – Exibir o tipo de dado e o tamanho da imagem.

```
# Exibe o tipo dos dados.
print(im.dtype)

# Exibe o tamanho da imagem.
print(im.shape)
```

Resultado:

```
uint8
(2600, 3914, 3)
```

Etapa 3 – Salvar a imagem.

```
# Salva a imagem.
cv2.imwrite('panda.png',im)
```

Resultado:

```
True
```

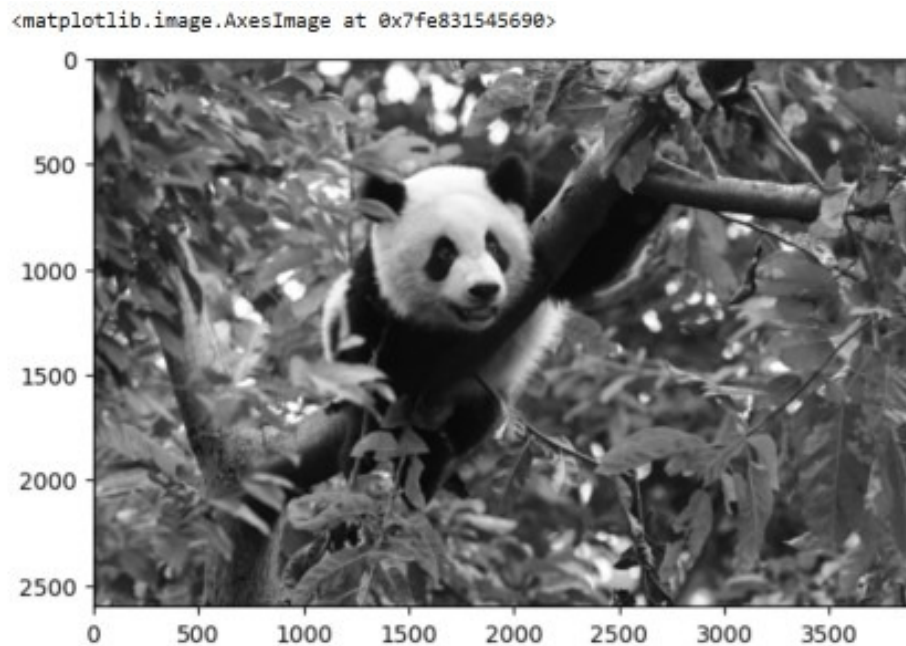
1.4.2 Conversão de Espaço de Cores

Etapa 1 – Converter imagem colorida para escala de cinza.

```
im = cv2.imread(r"panda.jpg")

# Conversão BGR para escala de cinza.
img_gray = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2GRAY
)
plt.imshow(img_gray, 'gray')
```

Resultado:

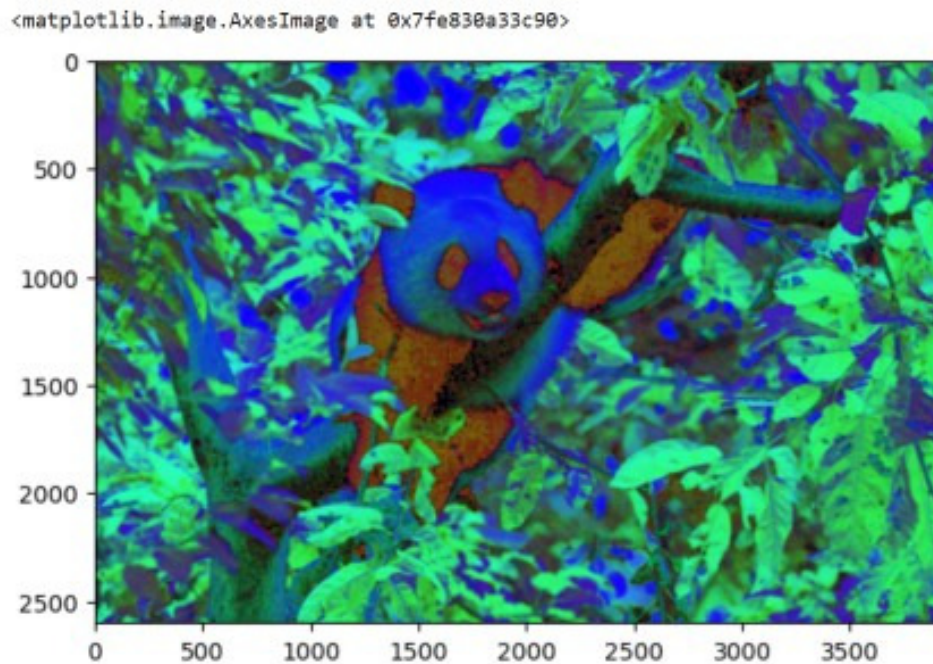


Etapa 2 – Converter de BGR para HSV.

```
im = cv2.imread(r"panda.jpg")

# Conversão BGR para HSV.
im_hsv = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2HSV
)
# O imshow assume RGB.
# Portanto, pode ocorrer distorção
# ao exibir HSV diretamente.
plt.imshow(im_hsv)
```

Resultado:



1.4.3 Transformações Geométricas

Etapa 1 – Aplicar translação na imagem.

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

%matplotlib inline

# Define a função de translação.
def translate(img, x, y):
    # Obtém o tamanho da imagem.
    (h, w) = img.shape[:2]
    # Define a matriz de translação.
    M = np.float32([
        [1, 0, x],
        [0, 1, y]
    ])
    # Executa a transformação afim.
    shifted = cv2.warpAffine(
        img,
        M,
        (w, h)
    )
    return shifted

# Carrega a imagem.
im = cv2.imread('panda.jpg')
```

```
im_RGB = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2RGB
)
plt.figure(figsize=(8,8))
plt.subplot(2,2,1)
plt.title('original')
plt.imshow(im_RGB)

# Move 500 pixels para baixo.
shifted_1 = translate(im_RGB, 0, 500)

plt.subplot(2,2,2)
plt.title('down move 500 pixels')
plt.imshow(shifted_1)

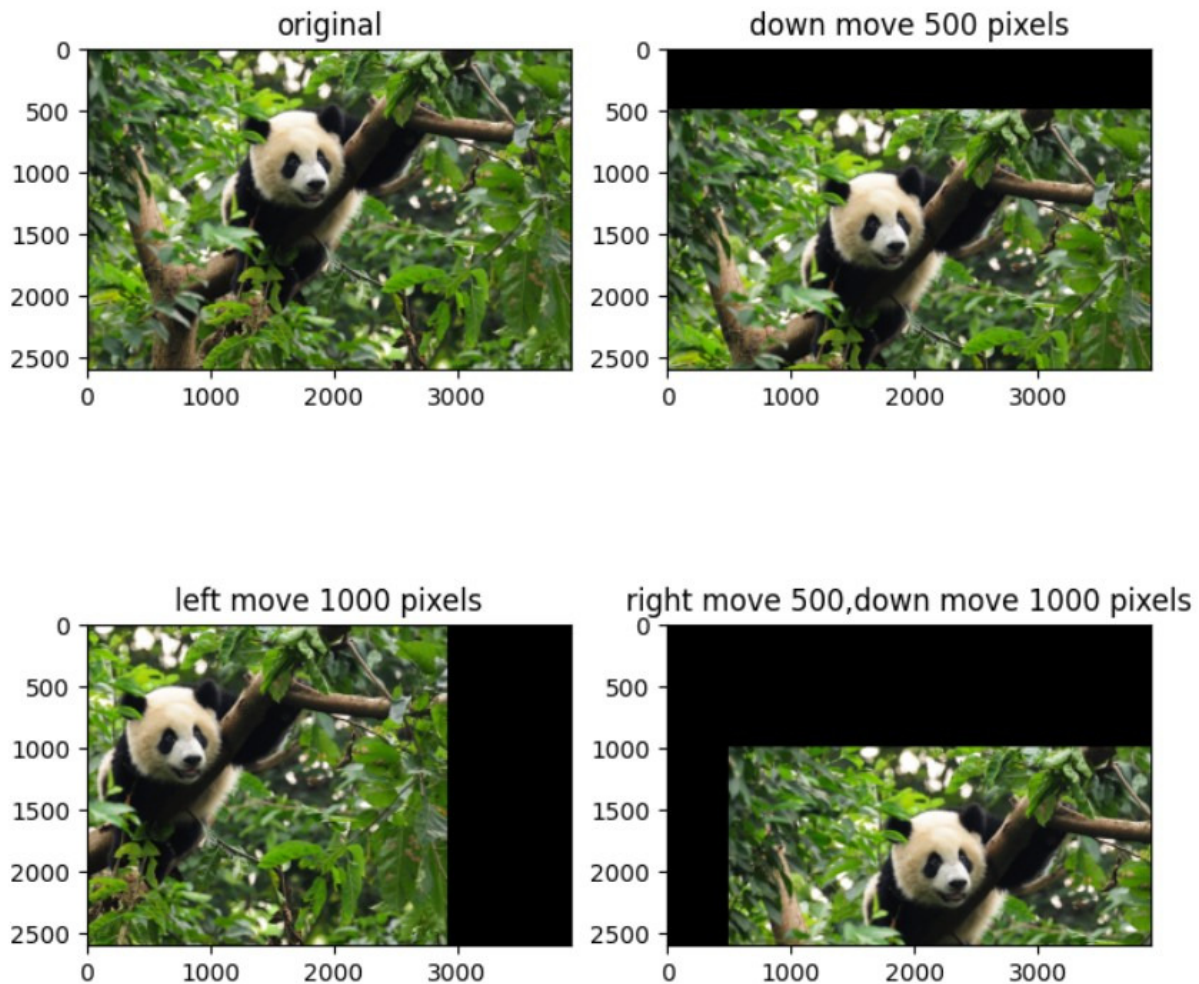
# Move 1000 pixels para esquerda.
shifted_2 = translate(im_RGB, -1000, 0)
plt.subplot(2,2,3)
plt.title('left move 1000 pixels')
plt.imshow(shifted_2)

# Move 500 pixels para direita
# e 1000 pixels para baixo.
shifted_3 = translate(
    im_RGB,
    500,
    1000
)

plt.subplot(2,2,4)
plt.title(
    'right move 500,down move 1000 pixels'
)
plt.imshow(shifted_3)
```

Resultado:


```
(<AxesSubplot:title={'center':'right move 500,down move 1000 pixels'}>,
Text(0.5, 1.0, 'right move 500,down move 1000 pixels'),
<matplotlib.image.AxesImage at 0x7fe830418c90>)
```



Etapa 2 – Aplicar rotação na imagem.

```
# Define a função de rotação.
def rotate(
    img,
    angle,
    center=None,
    scale=1.0
):
    # Obtém o tamanho da imagem.
    (h, w) = img.shape[:2]

    # Caso o centro não seja informado,
    # utiliza o centro da imagem.
    if center is None:
        center = (w / 2, h / 2)
```

```
# Calcula a matriz de rotação.
M = cv2.getRotationMatrix2D(
    center,
    angle,
    scale
)
# Executa a transformação afim.
rotated = cv2.warpAffine(
    img,
    M,
    (w, h)
)
return rotated

im = cv2.imread('panda.jpg')
im_RGB = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2RGB
)
plt.figure(figsize=(8,8))
plt.subplot(2,2,1)
plt.title('original')
plt.imshow(im_RGB)

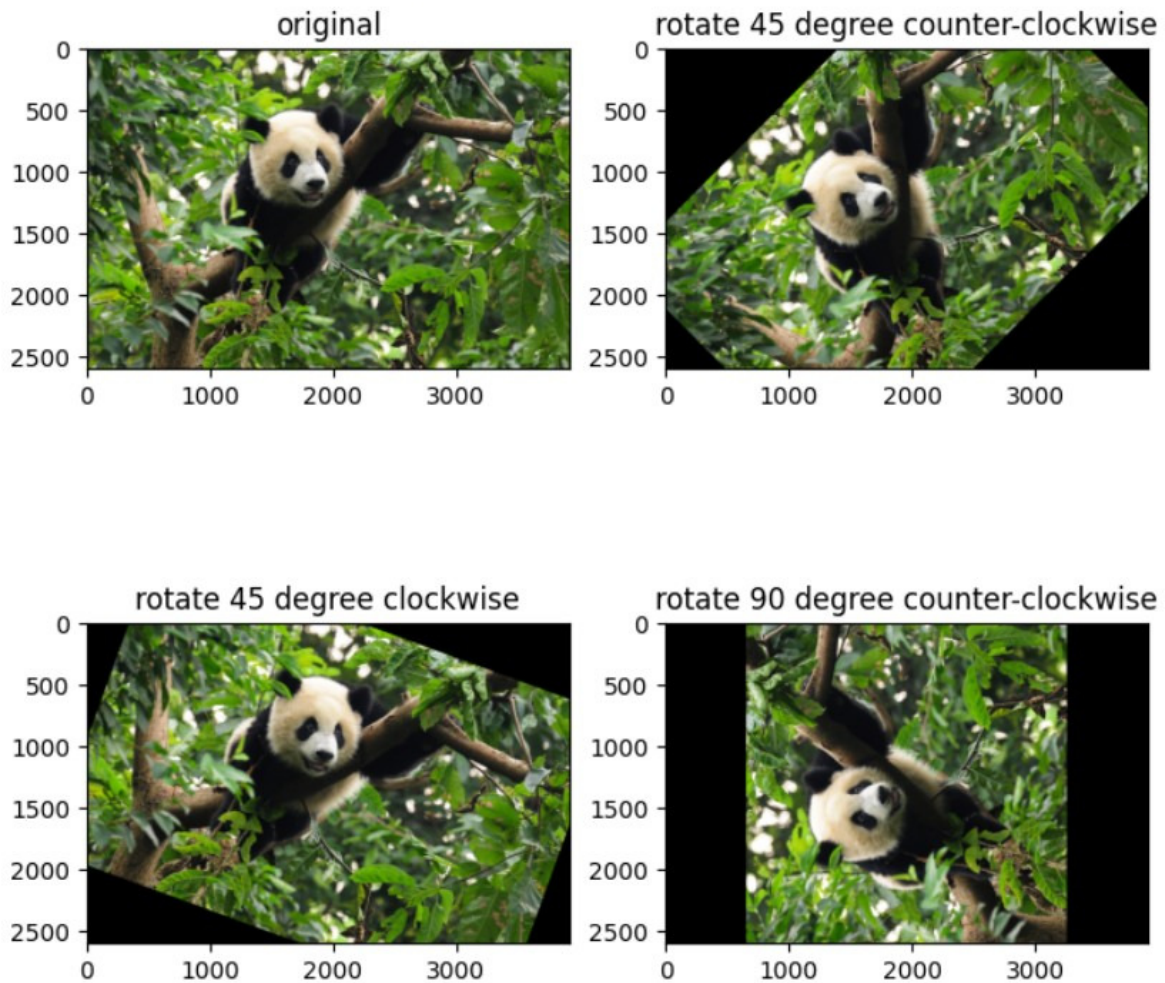
# Rotação de 45 graus anti-horário.
rotated_45 = rotate(im_RGB, 45)
plt.subplot(2,2,2)
plt.title(
    'rotate 45 degree counter-clockwise'
)
plt.imshow(rotated_45)

# Rotação de 20 graus horário.
rotated_minus20 = rotate(im_RGB, -20)
plt.subplot(2,2,3)
plt.title(
    'rotate 45 degree clockwise'
)
plt.imshow(rotated_minus20)

# Rotação de 90 graus anti-horário.
rotated_90 = rotate(im_RGB, 90)
plt.subplot(2,2,4)
plt.title(
    'rotate 90 degree counter-clockwise'
)
plt.imshow(rotated_90)
```


Resultado:

```
(<AxesSubplot:title={'center':'rotate 90 degree counter-clockwise'}>,
Text(0.5, 1.0, 'rotate 90 degree counter-clockwise'),
<matplotlib.image.AxesImage at 0x7fe83025be90>)
```



Etapa 3 – Aplicar espelhamento na imagem.

```
im = cv2.imread('panda.jpg')
im_RBG = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2RGB
)
plt.figure(figsize=(8,8))
plt.subplot(2,2,1)
plt.title('original')
plt.imshow(im_RBG)

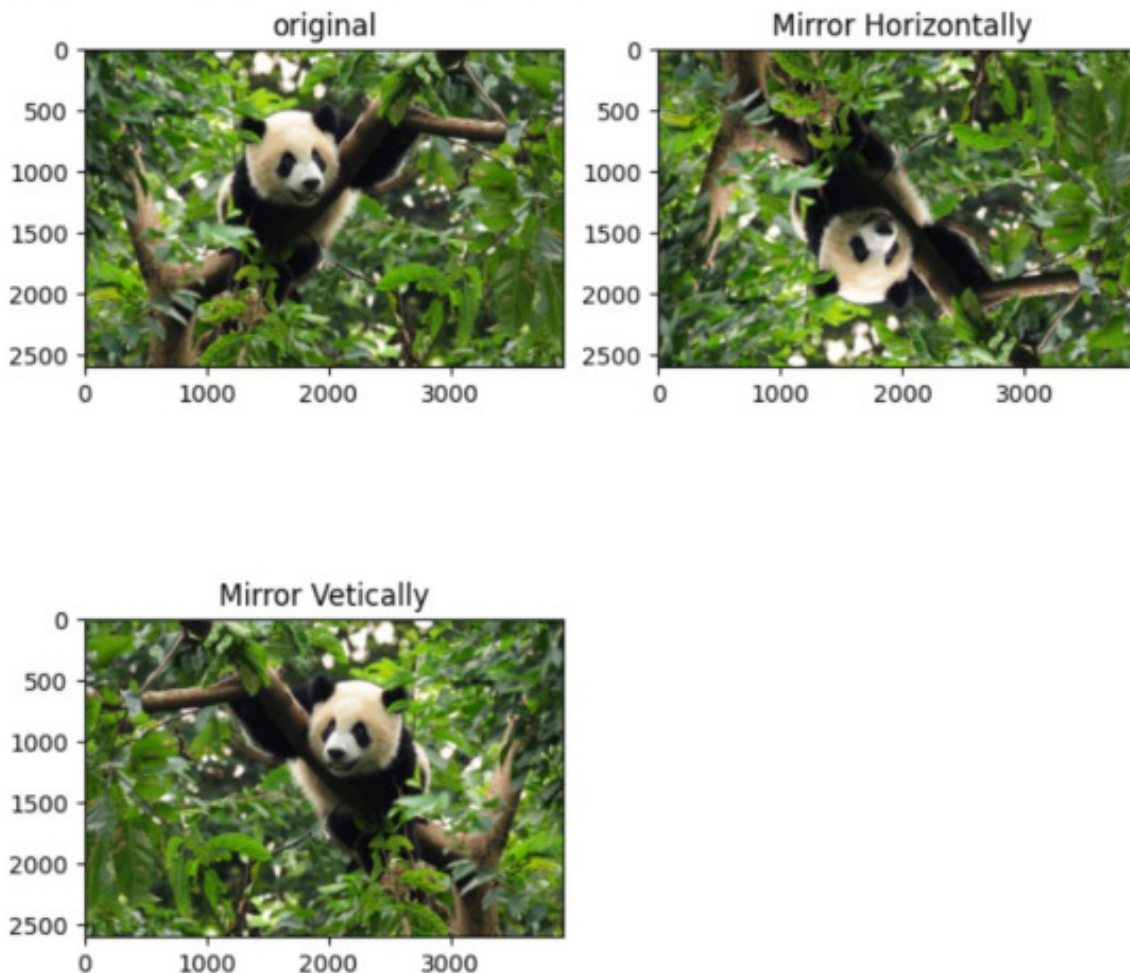
# Espelhamento horizontal.
im_flip0 = cv2.flip(im_RBG, 0)
plt.subplot(2,2,2)
```

```
plt.title('Mirror Horizontally')
plt.imshow(im_flip0)

# Espelhamento vertical.
im_flip1 = cv2.flip(im_RGB, 1)
plt.subplot(2,2,3)
plt.title('Mirror Vertically')
plt.imshow(im_flip1)
```

Resultado:

```
(<AxesSubplot:title={'center': 'Mirror Vetically'}>,
 Text(0.5, 1.0, 'Mirror Vetically'),
 <matplotlib.image.AxesImage at 0x7fe83012d810>)
```



Etapa 4 – Redimensionar a imagem.

```
im = cv2.imread('panda.jpg')
im_RGB = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2RGB
)
```

```
plt.figure(figsize=(8,8))
plt.subplot(2,2,1)
plt.title('original')
plt.imshow(im_RGB)

# Obtém o tamanho da imagem.
(h, w) = im_RGB.shape[:2]

# Define o novo tamanho.
dst_size = (2000,3000)

# Interpolação vizinho mais próximo.
method = cv2.INTER_NEAREST

# Redimensionamento.
resized = cv2.resize(
    im_RGB,
    dst_size,
    interpolation=method
)
plt.subplot(2,2,2)
plt.title('INTER_NEAREST')
plt.imshow(resized)

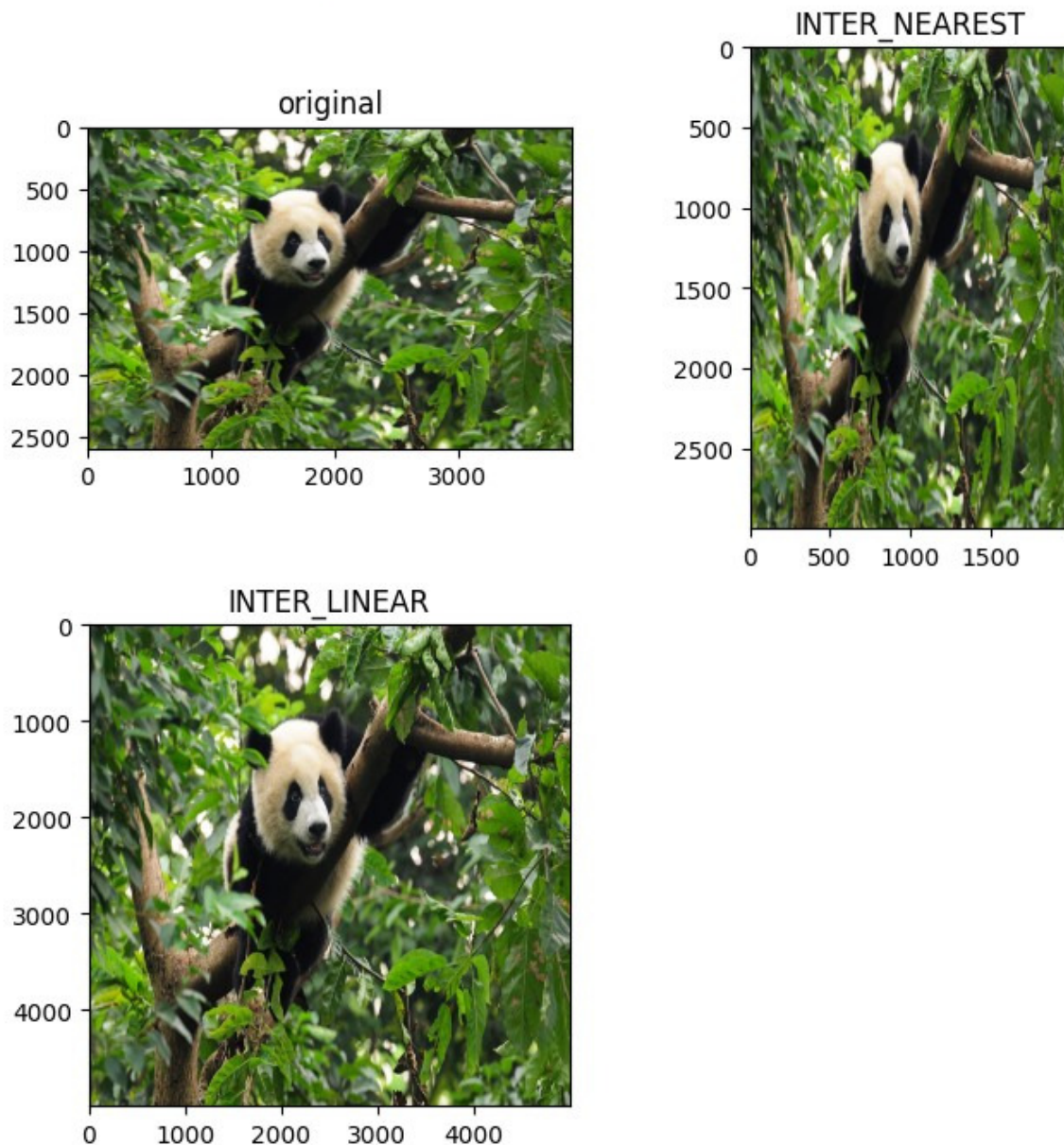
# Define novo tamanho.
dst_size = (5000,5000)

# Interpolação bilinear.
method = cv2.INTER_LINEAR

# Redimensionamento.
resized_linear = cv2.resize(
    im_RGB,
    dst_size,
    interpolation=method
)
plt.subplot(2,2,3)
plt.title('INTER_LINEAR')
plt.imshow(resized_linear)
```

Resultado:

```
(<AxesSubplot:title={'center':'INTER_LINEAR'}>,
Text(0.5, 1.0, 'INTER_LINEAR'),
<matplotlib.image.AxesImage at 0x7fe82fe79f10>)
```



1.4.4 Transformações em Escala de Cinza

Etapa 1 – Inversão, alongamento em tons de cinza, compressão em tons de cinza

```
import numpy as np
import cv2

from matplotlib import pyplot as plt

%matplotlib inline
```



```
# Define a função de transformação linear.
# Para k > 1:
# alongamento da escala de cinza.
#
# Para 0 < k < 1:
# compressão da escala de cinza.
#
# Para k = -1 e b = 255:
# inversão da imagem.
```

```
def linear_trans(img, k, b=0):
    # Cria a tabela de transformação.
    trans_list = [
        (np.float32(x)*k+b)
        for x in range(256)
    ]
    trans_table = np.array(trans_list)

    # Limita os valores entre 0 e 255.
    trans_table[trans_table>255] = 255
    trans_table[trans_table<0] = 0
    trans_table = np.round(
        trans_table
    ).astype(np.uint8)
    # Aplica a transformação.
    return cv2.LUT(img, trans_table)
```

```
im = cv2.imread(r'panda.jpg',0)
plt.figure(figsize=(8,8))
plt.subplot(2,2,1)
plt.title('original')
plt.imshow(im,'gray')
```

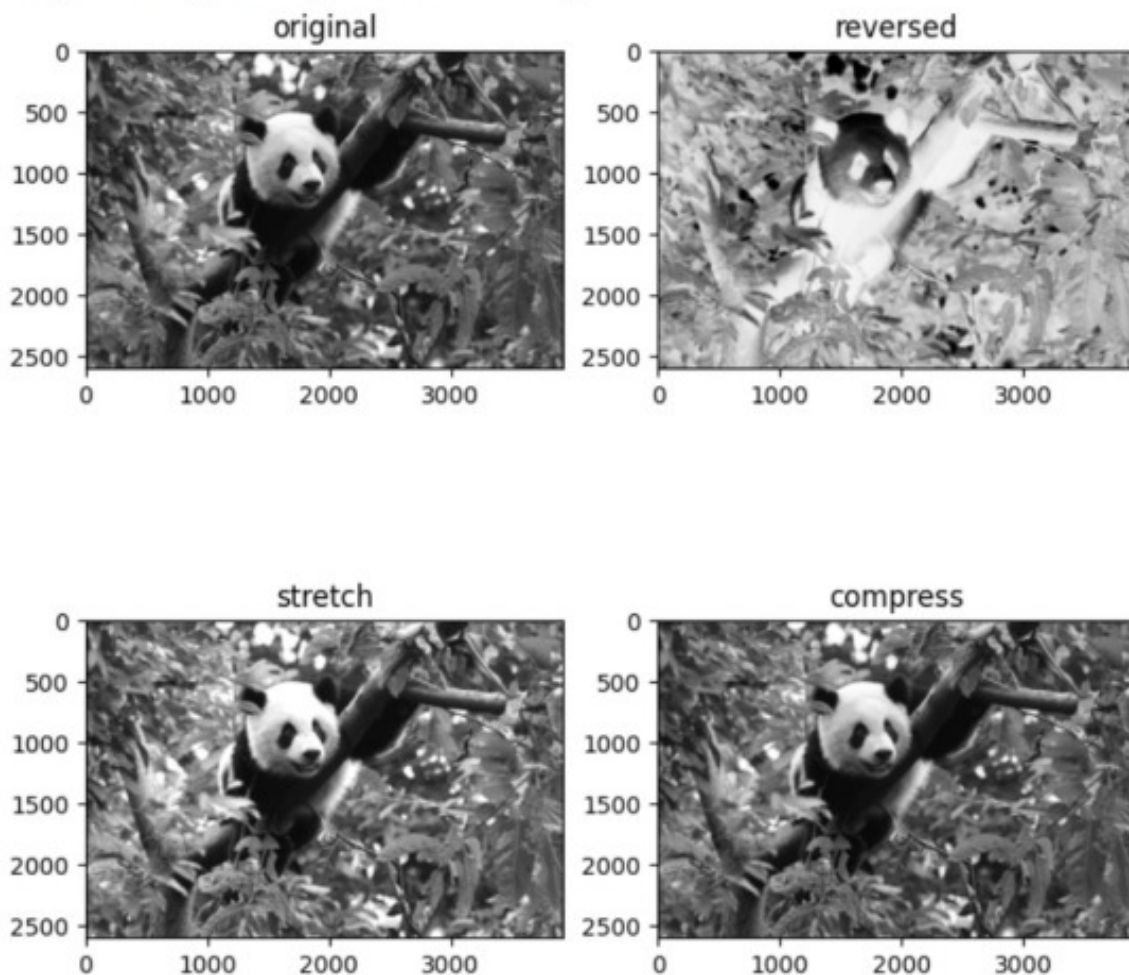
```
# Inversão.
im_inversion = linear_trans(
    im,
    -1,
    255
)
plt.subplot(2,2,2)
plt.title('reversed')
plt.imshow(im_inversion,'gray')
```

```
# Alongamento.
im_stretch = linear_trans(im, 1.2)
plt.subplot(2,2,3)
plt.title('stretch')
plt.imshow(im_stretch,'gray')
```

```
# Compressão.  
im_compress = linear_trans(im, 0.4)  
plt.subplot(2,2,4)  
plt.title('compress')  
plt.imshow(im_compress,'gray')
```

Resultado:

```
(<AxesSubplot:title={'center':'compress'}>,  
Text(0.5, 1.0, 'compress'),  
<matplotlib.image.AxesImage at 0x7fe82fc8c450>)
```



Etapa 2 – Transformação Gamma.

```
def gamma_trans(img, gamma):  
    # Normaliza para 1, realiza o cálculo gamma  
    # e depois restaura para [0,255].  
    gamma_list = [  
        np.power(x / 255.0, gamma) * 255.0  
        for x in range(256)  
    ]
```



```
# Converte a lista para np.array
# e define o tipo de dado como uint8.
gamma_table = np.round(
    np.array(gamma_list)
).astype(np.uint8)

# Usa a tabela de consulta do OpenCV
# para modificar os valores em escala de cinza.
return cv2.LUT(img, gamma_table)

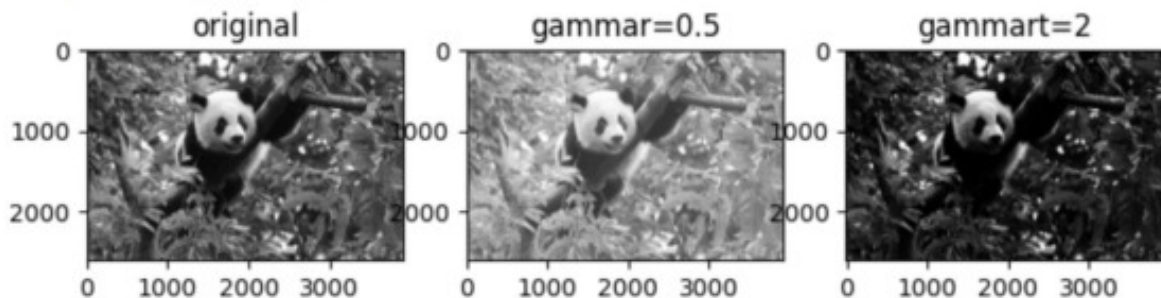
im = cv2.imread(r'panda.jpg',0)
plt.figure(figsize=(8,8))
plt.subplot(1,3,1)
plt.title('original')
plt.imshow(im,'gray')

# Utiliza gamma = 0.5 para realçar
# regiões escuras e comprimir regiões claras.
im_gamma05 = gamma_trans(im, 0.5)
plt.subplot(1,3,2)
plt.title('gamma=0.5')
plt.imshow(im_gamma05,'gray')

# Utiliza gamma = 2 para realçar
# regiões claras e comprimir regiões escuras.
im_gamma2 = gamma_trans(im, 2)
plt.subplot(1,3,3)
plt.title('gamma=2')
plt.imshow(im_gamma2,'gray')
```

Resultado:

```
(<AxesSubplot:title={'center':'gamma=2'}>,
 Text(0.5, 1.0, 'gamma=2'),
 <matplotlib.image.AxesImage at 0x7fe82fb0f4d0>)
```



Etapa 3 – Equalização de histograma.

```
import cv2
from matplotlib import pyplot as plt
```

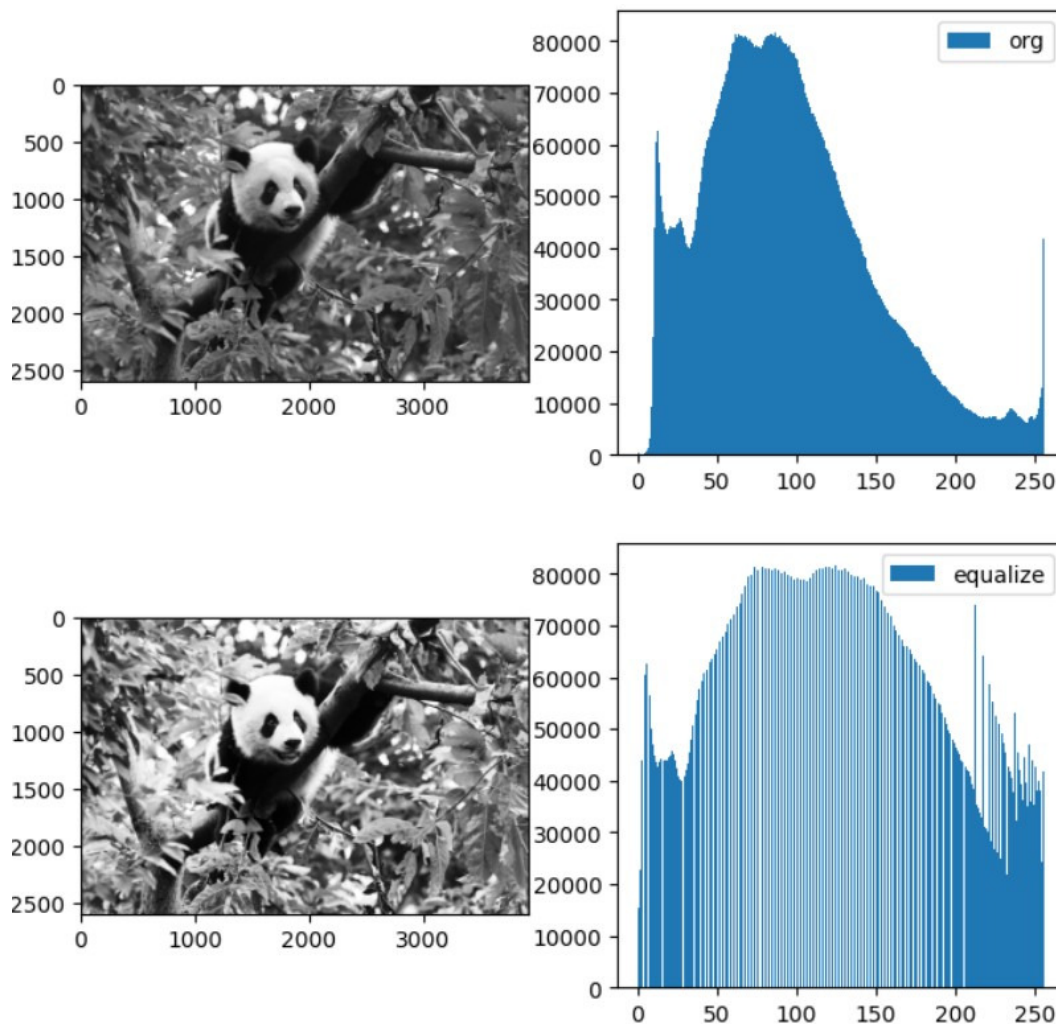
```
im = cv2.imread(r'panda.jpg',0)
plt.figure(figsize=(8,8))
plt.subplot(2,2,1)
plt.imshow(im,'gray')

# Chama a API de equalização
# de histograma do OpenCV.
im_equal = cv2.equalizeHist(im)
plt.subplot(2,2,3)
plt.imshow(im_equal,'gray')

# Exibe o histograma da imagem original.
plt.subplot(2,2,2)
plt.hist(
    im.ravel(),
    256,
    [0,256],
    label='org'
)
plt.legend()

# Exibe o histograma da imagem equalizada.
plt.subplot(2,2,4)
plt.hist(
    im_equal.ravel(),
    256,
    [0,256],
    label='equalize'
)
plt.legend()
plt.show()
```

Resultado:



Etapa 4 – Aplicar binarização na imagem.

Uma imagem inclui primeiro plano (alvo), fundo e ruído. Para extrair diretamente o alvo do primeiro plano de uma imagem digital, o método mais utilizado é a binarização.

Defina um limiar T e utilize-o para dividir os dados da imagem em duas partes:

- grupo de pixels maior que T ;
- grupo de pixels menor que T .

Esse método é conhecido como binarização de imagem.

O Python-OpenCV fornece a função:

```
cv2.threshold(src, x, y, method)
```

- **src**: imagem original, que deve ser em escala de cinza;
- **x**: limiar utilizado para classificação dos pixels;
- **y**: cor de preenchimento;

- `method`: método utilizado para binarização.

Essa função possui dois valores de retorno:

- `retVal`: limiar obtido;
- imagem binarizada.

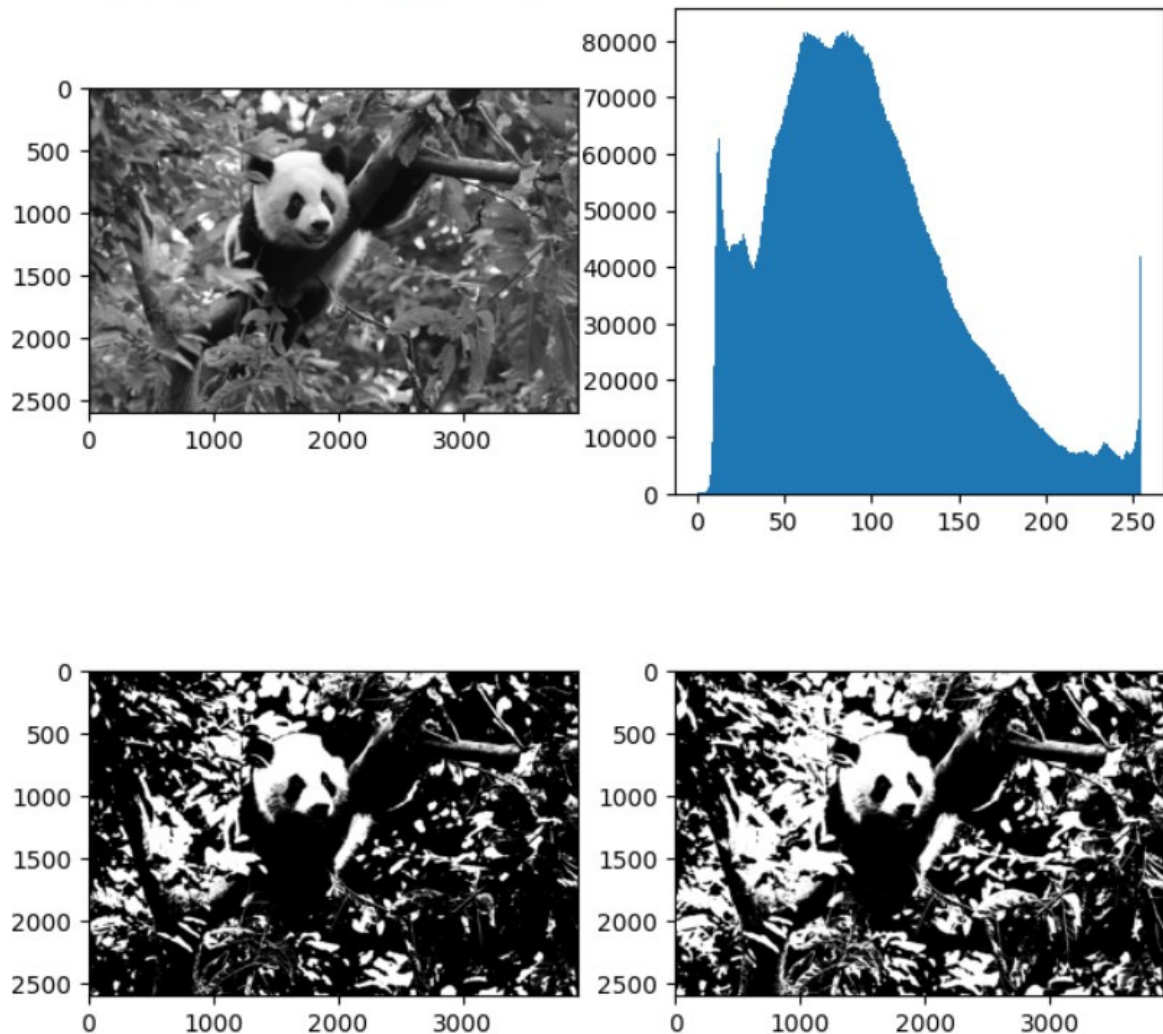
```
img = cv2.imread(
    r'panda.jpg',
    0
)
# Segmentação simples por limiar.
ret1,th1 = cv2.threshold(
    img,
    127,
    255,
    cv2.THRESH_BINARY
)
# Mé todo de Otsu.
ret2,th2 = cv2.threshold(
    img,
    0,
    255,
    cv2.THRESH_OTSU
)
print(
    'Simple threshold value:',
    ret1
)
print(
    'Optimal threshold value:',
    ret2
)
plt.figure(figsize=(8,8))
plt.subplot(221)
plt.imshow(img,'gray')
plt.subplot(222)

# O mé todo ravel converte
# a matriz em vetor unidimensional.
plt.hist(img.ravel(),256)
plt.subplot(223)
plt.imshow(th1,'gray')
plt.subplot(224)
plt.imshow(th2,'gray')
```

Resultado:

Simple threshold value: 127.0
Optimal threshold value: 108.0

(<AxesSubplot:>, <matplotlib.image.AxesImage at 0x7fe8241c2b10>)



O experimento compara o método de limiarização de Otsu (OTSU) com o método manual geral, demonstrando os efeitos da segmentação da imagem.

1.4.5 Processamento Morfológico

Processe imagens binárias utilizando métodos morfológicos.

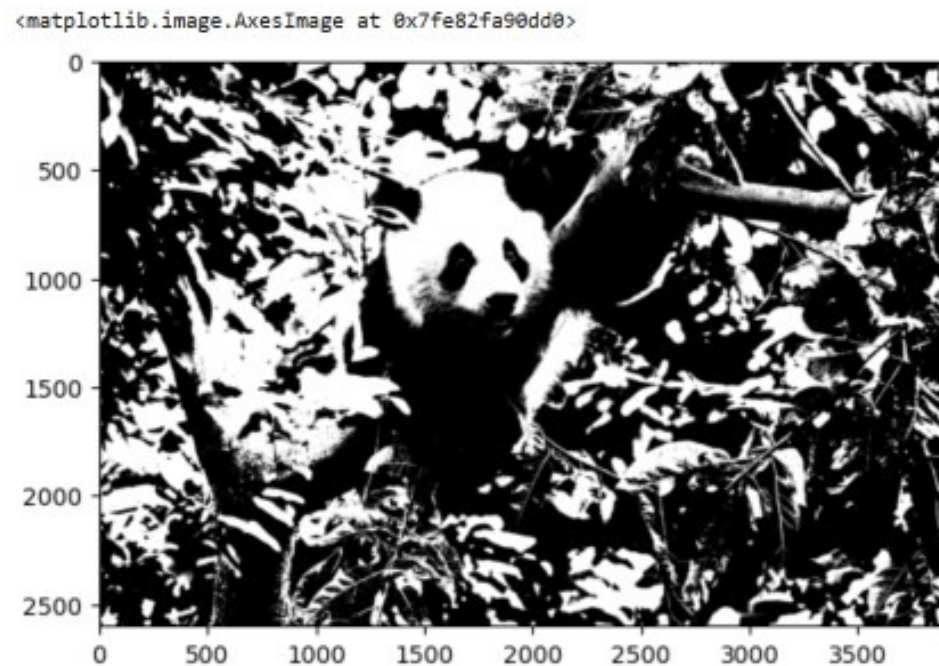
1. Os alvos extraídos do primeiro plano geralmente são definidos como branco, enquanto o fundo é preto. Caso contrário, utilize a função `cv2.bitwise_not()` para inverter as cores da imagem binária.
2. Observe inicialmente os alvos e ruídos da imagem e escolha uma combinação adequada de operações morfológicas.

- “Dilatação” expande as regiões destacadas da imagem, aumentando as áreas brancas;
- “Erosão” reduz as regiões destacadas da imagem, aumentando as áreas pretas.

Etapa 1 – Ler e binarizar a imagem.

```
img = cv2.imread(  
    'panda.jpg',  
    0  
)  
_, bin_img = cv2.threshold(  
    img,  
    0,  
    255,  
    cv2.THRESH_OTSU  
)  
plt.imshow(bin_img, 'gray')
```

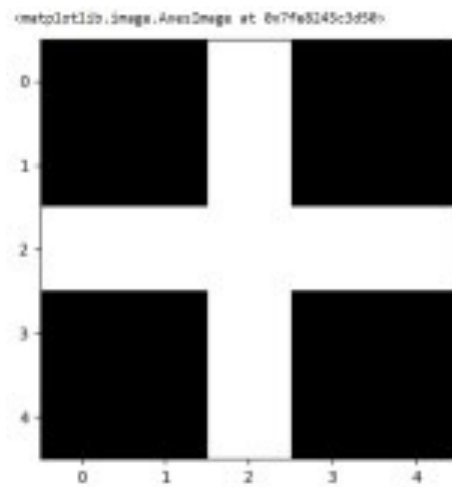
Resultado:



Etapa 2 – Definir elementos estruturantes.

```
# Define os elementos estruturantes.  
element = cv2.getStructuringElement(  
    cv2.MORPH_CROSS,  
    (5,5)  
)  
plt.imshow(element, 'gray')
```

Resultado:



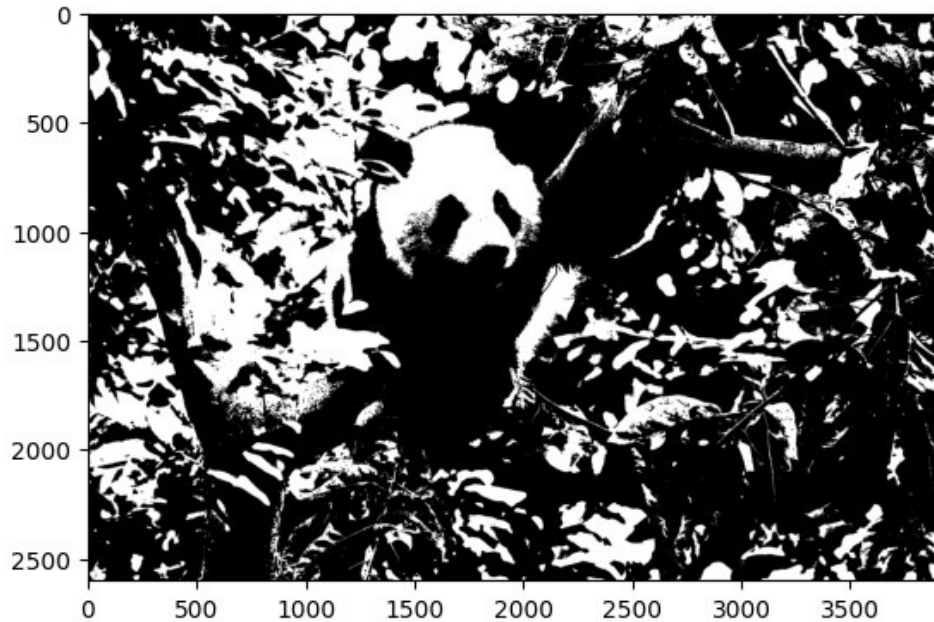
Etapa 3 – Erosão.

```
# Realiza erosão da imagem.
eroded = cv2.erode(
    bin_img,
    element
)
# Exibe a imagem erodida.
plt.imshow(eroded,'gray')

# Imagem original.
plt.figure()
plt.imshow(bin_img,'gray')
```

Resultado:

<matplotlib.image.AxesImage at 0x7fe8245a0910>



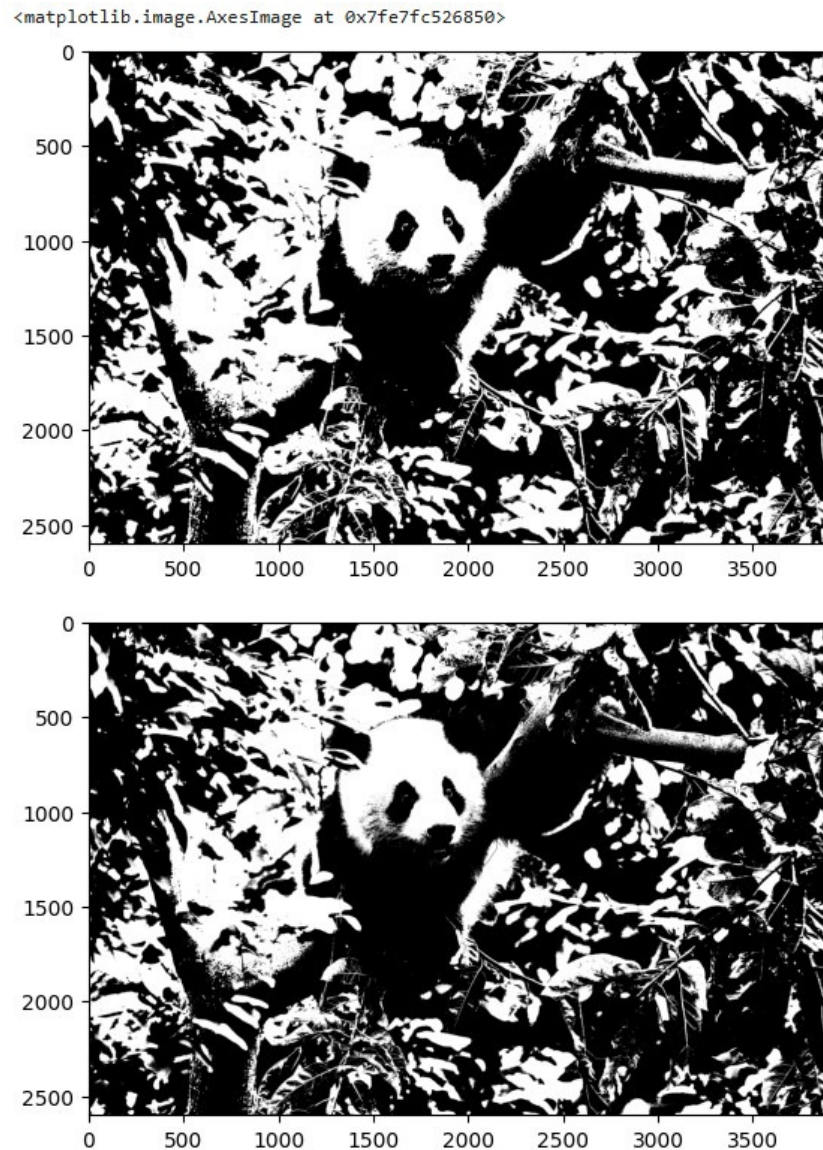
Etapa 4 – Dilatação.

```
# Realiza dilatação da imagem.
dilated = cv2.dilate(
    bin_img,
    element
)

# Exibe a imagem dilatada.
plt.imshow(dilated, 'gray')
```

```
# Imagem original.  
plt.figure()  
plt.imshow(bin_img, 'gray')
```

Resultado:



1.4.6 Filtragem de Imagens

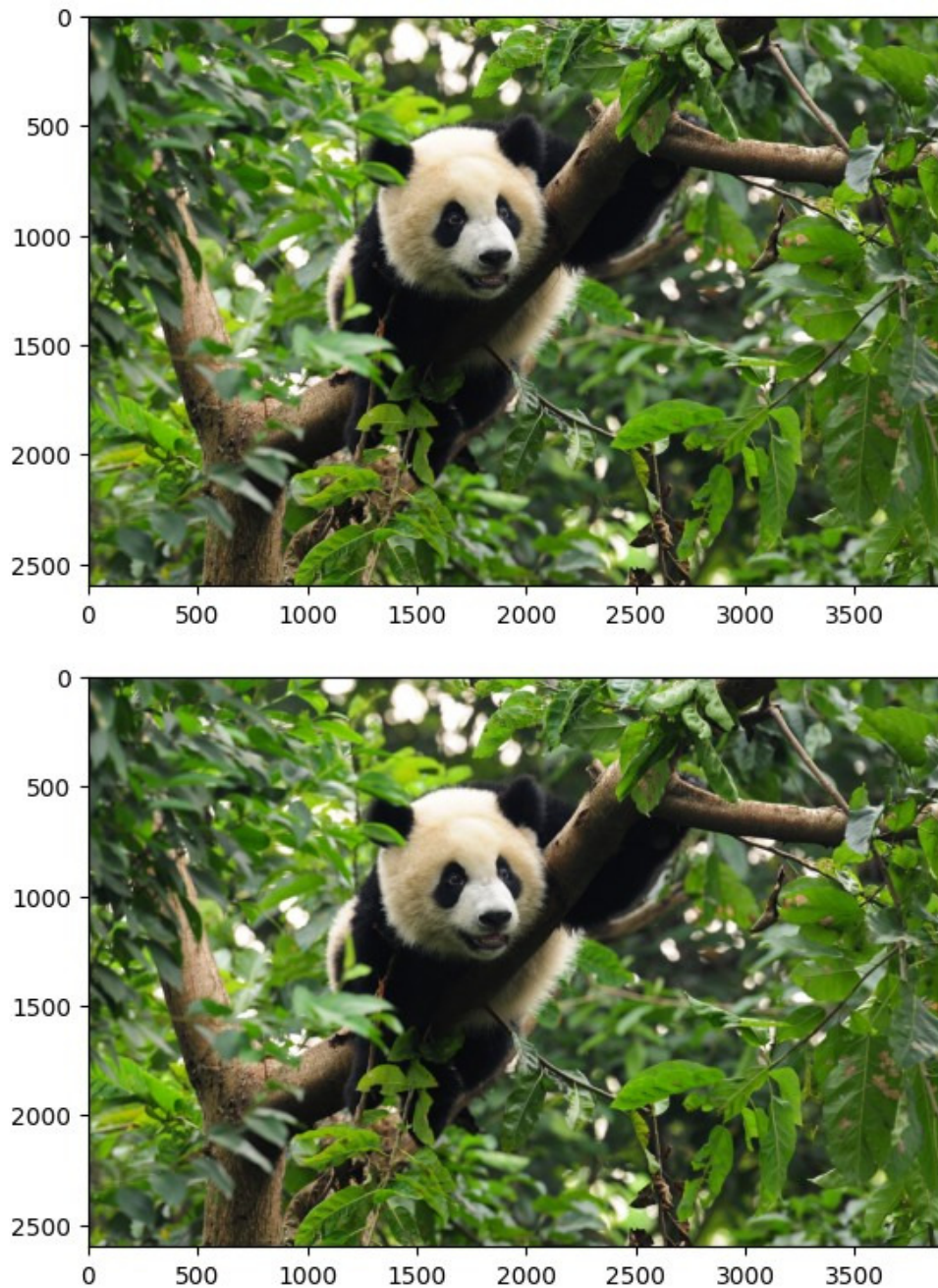
Etapas 1 – Aplicar filtro mediana.

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
  
%matplotlib inline
```

```
im = cv2.imread('panda.jpg')
im = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2RGB
)
plt.imshow(im)

# Chama a API de filtro mediana
# do OpenCV.
im_medianblur = cv2.medianBlur(
    im,
    5
)
plt.figure()
plt.imshow(im_medianblur)
plt.show()
```

Resultado:



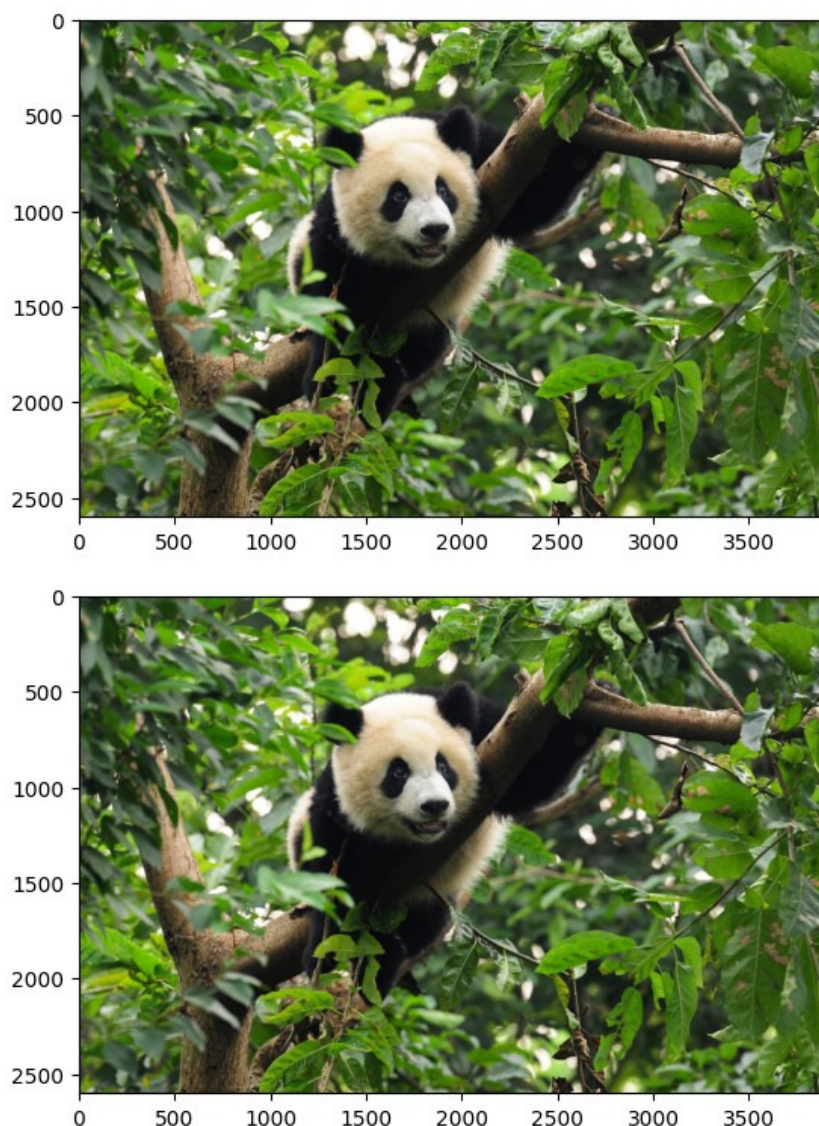
Etapa 2 – Aplicar filtro média.

```
im = cv2.imread('panda.jpg')
im = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2RGB
)
plt.imshow(im)

# Chama a API de filtro média
# do OpenCV.
```

```
im_meanblur = cv2.blur(  
    im,  
    (3,3)  
)  
plt.figure()  
plt.imshow(im_meanblur)  
plt.show()
```

Resultado:



Etapa 3 – Aplicar filtro Gaussiano.

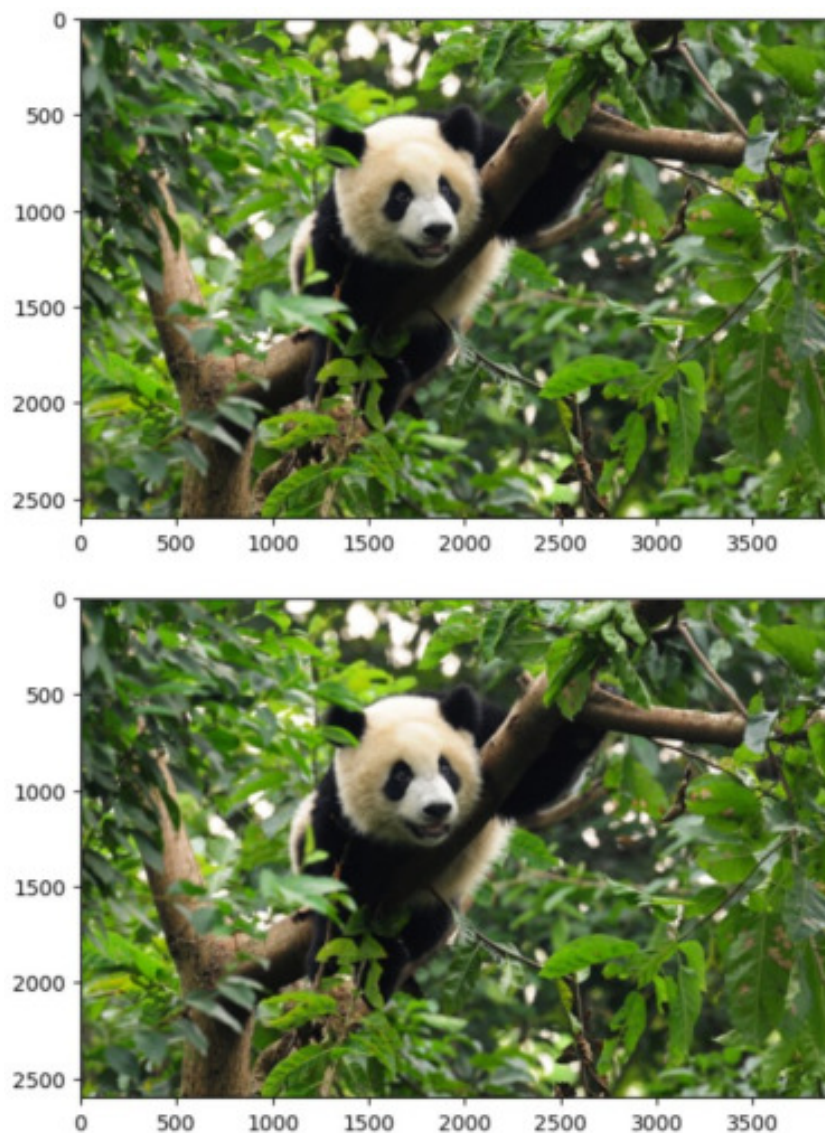
```
im = cv2.imread('panda.jpg')  
im = cv2.cvtColor(  
    im,  
    cv2.COLOR_BGR2RGB  
)
```



```
plt.imshow(im)

# Chama a API de filtro Gaussiano
# do OpenCV.
im_gaussianblur = cv2.GaussianBlur(
    im,
    (5,5),
    0
)
plt.figure()
plt.imshow(im_gaussianblur)
plt.show()
```

Resultado:



Etapa 4 – Aplicar nitidez na imagem.

```
im = cv2.imread('panda.jpg')
im = cv2.cvtColor(
    im,
    cv2.COLOR_BGR2RGB
)
plt.imshow(im)

# Operador de nitidez.
sharpen_1 = np.array([
    [-1,-1,-1],
    [-1,9,-1],
    [-1,-1,-1]
])

# Usa filter2D para filtragem.
im_sharpen1 = cv2.filter2D(
    im,
    -1,
    sharpen_1
)
plt.figure()
plt.imshow(im_sharpen1)

# Operador de nitidez 2.
sharpen_2 = np.array([
    [0,-1,0],
    [-1,8,-1],
    [0,1,0]
]) / 4.0

# Usa filter2D para filtragem.
im_sharpen2 = cv2.filter2D(
    im,
    -1,
    sharpen_2
)
plt.figure()
plt.imshow(im_sharpen2)
```

Resultado:

<matplotlib.image.AxesImage at 0x7f745027b250>





1.5 Resumo

Este experimento apresentou as operações específicas de experimentos de pré-processamento de imagens utilizando a biblioteca **OpenCV** baseada em **Python**. O uso da biblioteca **OpenCV** foi demonstrado para realizar operações básicas com imagens, conversão de espaço de cores, transformações geométricas, transformações em escala de cinza, processamento morfológico e filtragem de imagens, aprimorando a compreensão das técnicas de pré-processamento de imagens e fornecendo orientação prática para utilização dessas técnicas.

1.6 Questões

1. Quais são as diferenças entre o método de Otsu e o método manual de limiarização na binarização de imagens?
2. O método de Otsu pode selecionar automaticamente o limiar para obter melhores resultados de segmentação de imagens.

Referências

1. HUAWEI TECHNOLOGIES CO., LTD. **Artificial Intelligence Technology and Application: Python Lab Guide**. Huawei Technologies Co., Ltd., China.
2. David Beazley and Guido Van Rossum. **Python: Essential Reference**. *New Riders Publishing*, USA, 1999.